ECHO

A Platform for Data Flow Integration

- Lightweight API Integration Platform
- Easy to use, with a visual, zero-code builder
- From Jira straight to deployment
- Interface-spec-driven design (OpenAPI first)
- Built 100% in-house: every component is well understood, so changes and new features are quick
- No licensing fees
- Full source code ownership
- Built on proven, industry-tested libraries
- Modular design: parts are easy to swap and customize

Using ECHO

Performed by the Subject Matter Expert

01 Straight from the subject matter expert to deployment

02 Drag-and-drop design throughout

03 Code generated behind the scenes

04 No procedural code in Java or any other programming language

05 Every use case can be tested directly from the web-based GUI

Typical Steps of a Flow’s Development

a

Create a user story with sample northside requests and responses (defines the request and response payloads, plus query parameters)

b

Identify the downstream calls to make (one or many)

c (Optional)

Check whether the downstream calls can run in parallel to save execution time, or whether they depend on each other

d

Map data from the northside request to each downstream call

e

Map data from each downstream call to the final response

> f. These steps are not limited to HTTP-based REST calls

Demo

JIRA USER STORY

GNTSNETDEV-15329

REQUEST

`GET /Inventory/v1/Resource/node?id=AVDLAZRH4P001`

RESPONSE

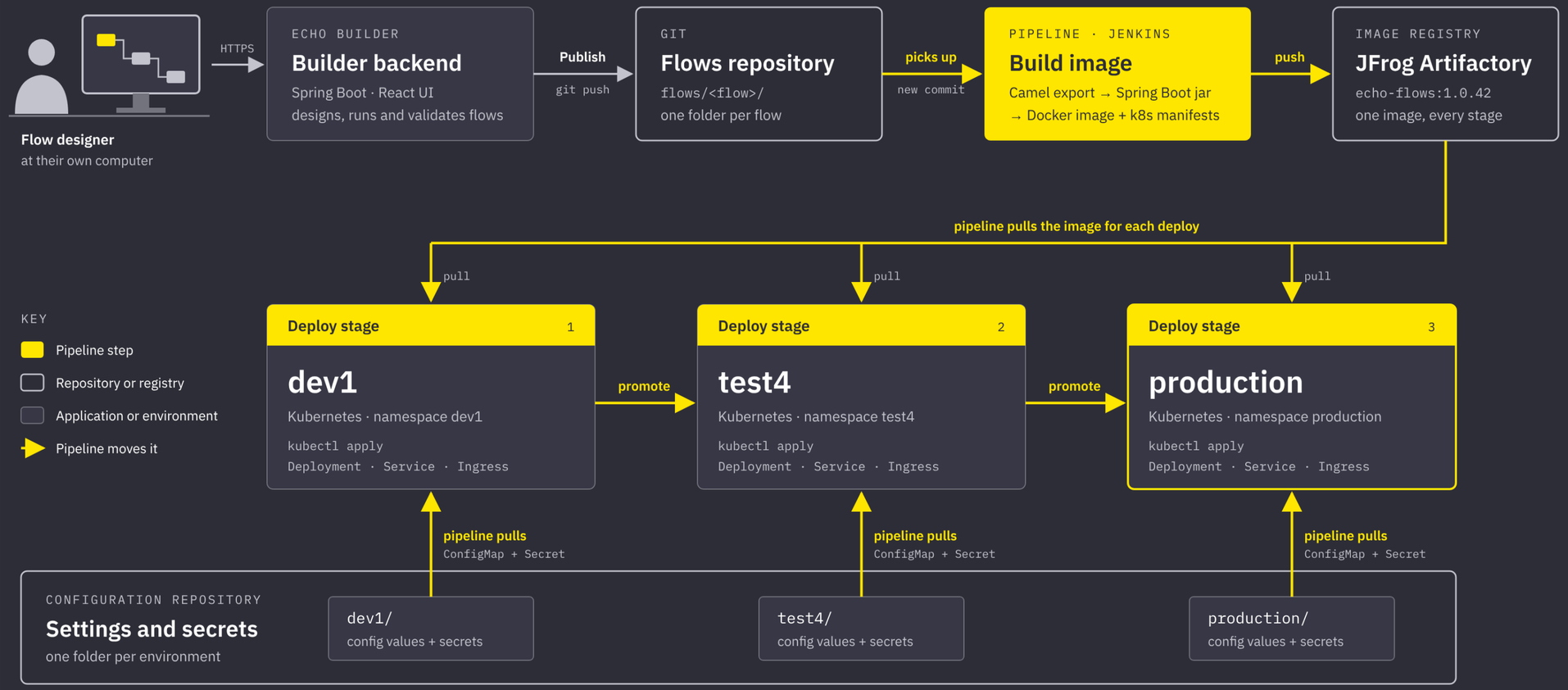
Shown on the right


```
{
  "rack": null,
  "nodeDef": "ADVA FSP 150-GE114Pro 1G NID (4P)",
  "description": "FSP 150CC-GE114Pro, I-Temp, with
integrated AC PSU...",
  "name": "AVDLAZRH4P001",
  "type": "NE::ADVA FSP 150-GE114Pro 1G NID (4P)",
  "provisionStatus": "In Service",
  "functionalStatus": "Not Specified",
  "locationIdentifier": {
    "internalIdentifier": "2639123",
    "entityType": "Location", "id": "AVDLAZRH" },
  "identifier": {
    "internalIdentifier": "3164712",
    "entityType": "Node", "id": "AVDLAZRH4P001" },
  "extensionAttributes": { "item": [
    {"name": "PSEUDO_ENABLED", "value": "Yes"},
    {"name": "LOGICAL_STATUS", "value": "Planned"},
    {"name": "EQUIPMENT_CODE", "value": "4P"},
    {"name": "NETWORK_NAME", "value": "AVDLAZRH4P001"},
    {"name": "ROLECODE", "value": "NID"},
    {"name": "ISNID", "value": "Yes"}
  ] }
}
```

Built-in Features, Out of the Box

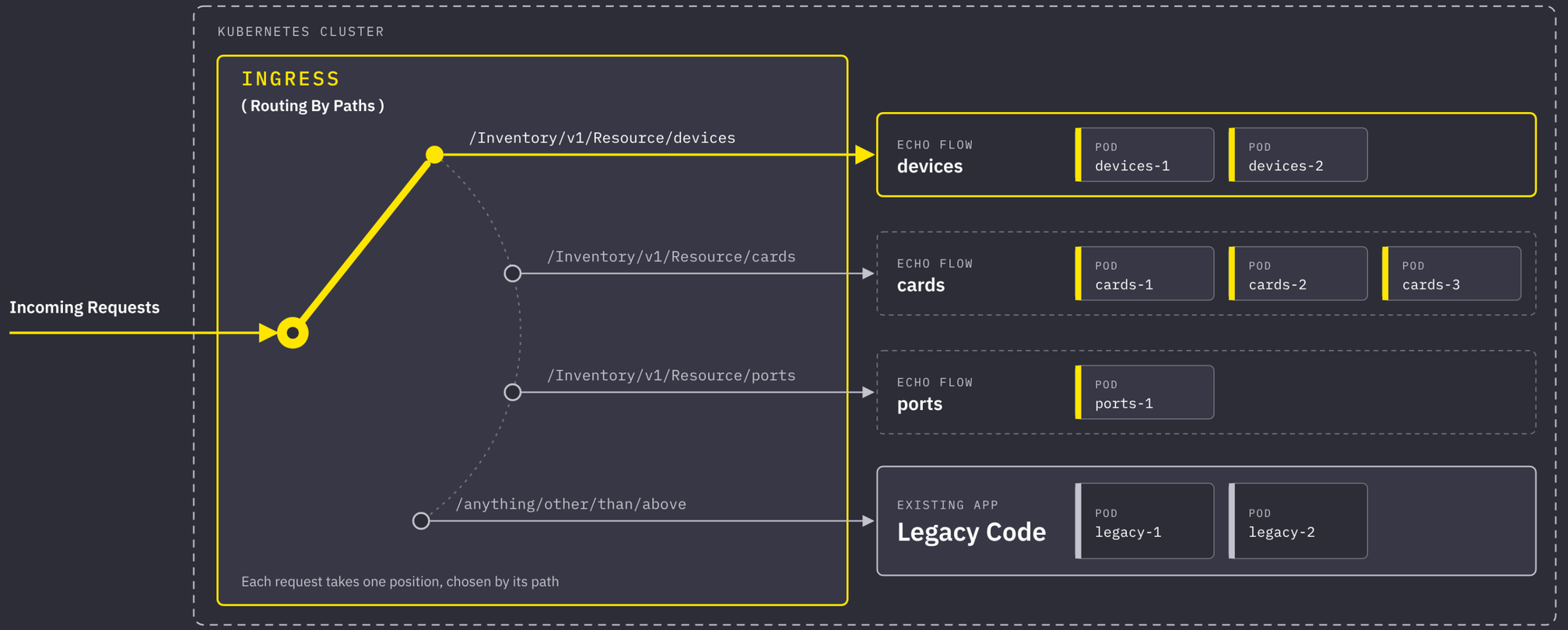
Validation	Requests that don’t match the interface spec (built from the sample payloads) are rejected with an error that gives the exact reason
Logging	Every incoming request and every downstream API call, in ECS format
Swagger UI	A page that always describes every flow, including “Try it out”
Version history	Every published version is stored in Git and can be restored at any time
Observability	Observability and traceability with Prometheus metrics and OpenTelemetry tracing

Technical Details: The DevOps Pipeline



The image holds no environment values. Each deploy stage reads that environment's settings and secrets from the configuration repository, so one image moves from dev1 to test4 to production unchanged.

Request Routing Using Ingress



ECHO adds Ingress rules only for its flows' paths, so every other path still reaches the legacy application.

The ECHO platform has five parts:

a



Builder Frontend

Runs in the user's browser: flows are designed, tested and published here.

b



Builder Backend

A Docker image running Java 21 that serves the front end. Not part of the daily build and deployment.

c



Pipeline

The scripts needed to build the runtime flows are provided.

d



Kubernetes Ingress

Configuration files

e



ECHO Runtime

Docker images running Java 21 that contain the flows. Bundle mode: all flows in one image; per-flow: one per flow.

Can be deployed on standalone servers, but ECHO includes features designed for Kubernetes

Can run side by side with legacy deployments, using Kubernetes Ingress routing

Can be deployed in bundle mode (all flows in one pod) or per-flow mode (each flow in its own pod)

Technical Details: Internals

Area	Technology
Builder front end	React and PatternFly, with Kaoto (flow designer) and Apicurio (OpenAPI designer) embedded
Builder back end	Java 21 and Spring Boot, with an OpenAI-compatible AI engine for the Jira import
Runtime	Apache Camel and Spring Boot, with Prometheus and OpenTelemetry
Build and deploy	Docker images and Kubernetes manifests, with Playwright for automated UI tests
Flows	Every flow is plain text in the Camel YAML DSL (Domain-Specific Language)